

Introduction to Algorithm

기초 알고리즘

다이나믹 프로그래밍 기초 예제



다이나믹 프로그래밍의 원리는 쉽지만,
문제를 풀어보려고 하면 생각보다 아이디어를 떠올리기 어려움



이는 어떤 문제를 다이나믹 프로그래밍으로 해결해야 할 지
파악하기도 쉽지 않고, 점화식을 세우는 것도 어렵기 때문



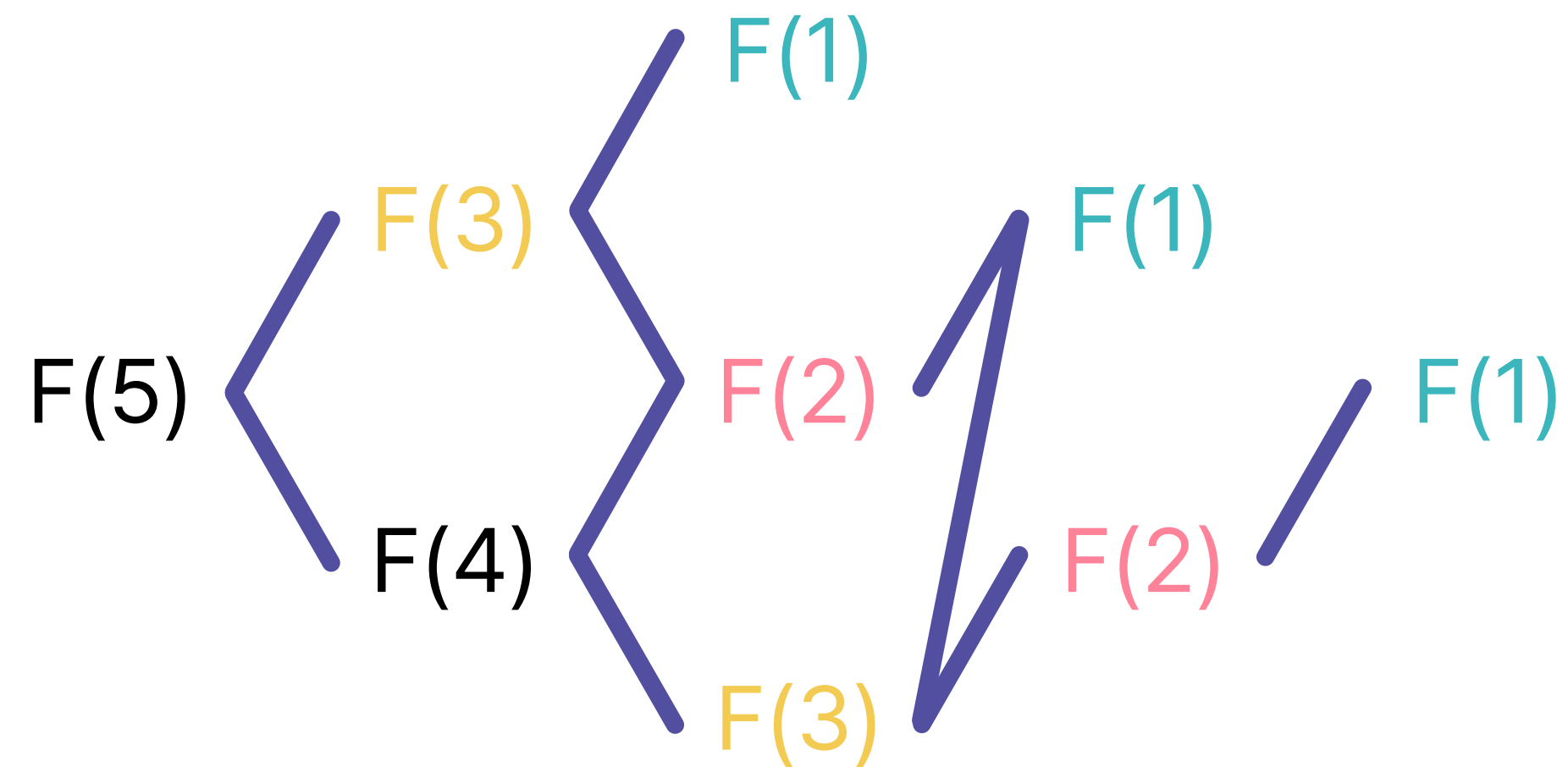
따라서, 다양한 문제들을 통해 다이나믹 프로그래밍 문제를
해결하는 방법을 충분히 연습하도록 하자



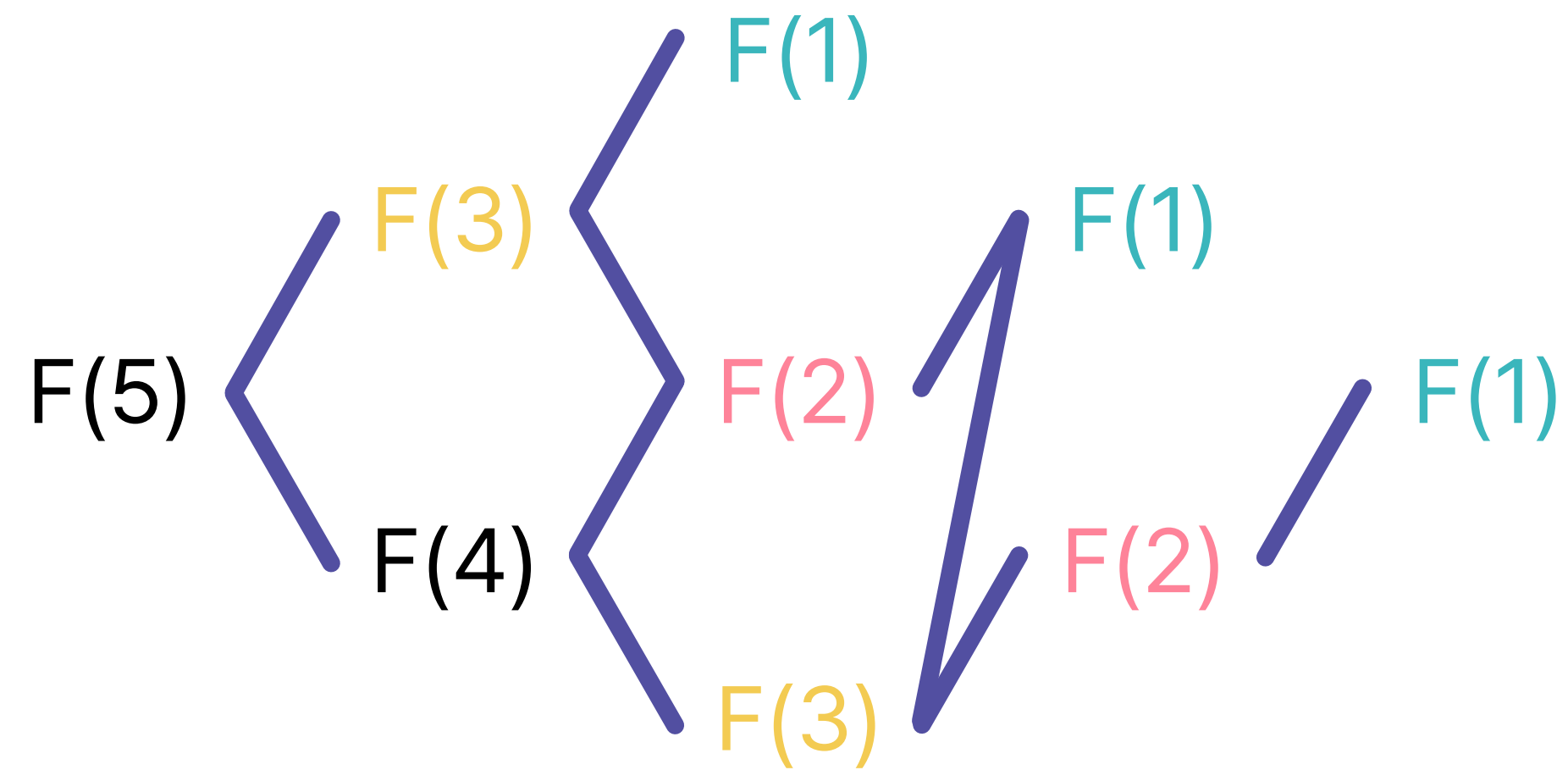
먼저, 어떤 문제를 다이나믹 프로그래밍으로
해결할 수 있을까?



동적계획법의 조건



우리는 처음 다이나믹 프로그래밍을 공부했을 때,
피보나치 수를 활용했음



피보나치 수를 재귀함수 문제를 해결하려고 할 때,
중복된 부분이 너무 많아 비효율적이라고 하였음



동적계획법의 조건

```
def fibo(n):  
    fibonacci = [-1, 1, 1]  
    if n < 3: return 1  
    for i in range(3, n+1):  
        fibonacci.append(fibonacci[i-1] + fibonacci[i-2])  
    return fibonacci[n]
```

그리고 이 문제를 해결하기 위해,
다이나믹 프로그래밍을 활용했음



동적계획법의 조건

```
def fibo(n):  
    fibonacci = [-1, 1, 1]  
    if n < 3: return 1  
    for i in range(3, n+1):  
        fibonacci.append(fibonacci[i-1] + fibonacci[i-2])  
    return fibonacci[n]
```

결국, 우리는 다이나믹 프로그래밍을 사용하여
중복된 부분을 제거한 것



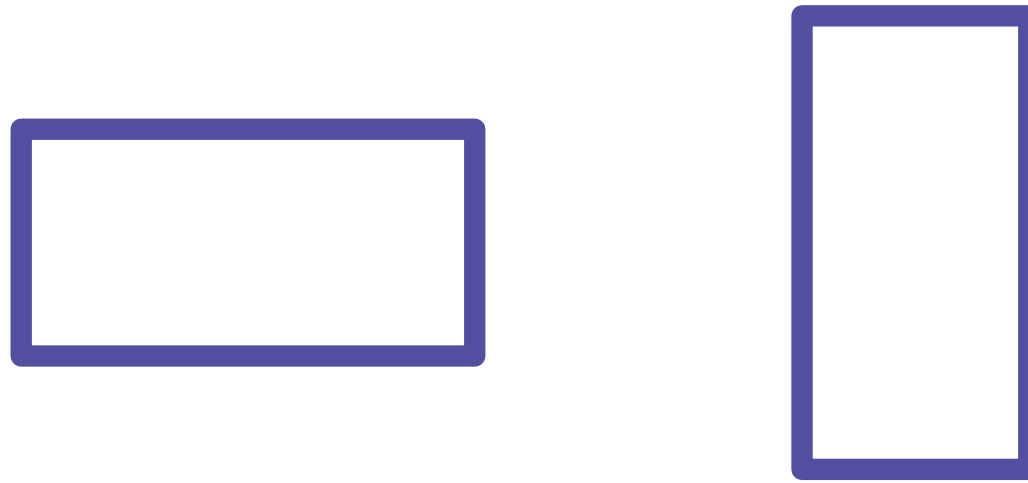
다시 말하자면, 다이나믹 프로그래밍은
점화식을 세울 수 있으며, 계산 과정에서 중복된 부분이
발생하는 경우에 사용할 수 있다고 생각하면 됨



그렇다면, 문제를 통해 실제로 이 조건을 어떻게
적용할 수 있을지 확인해 보도록 하자!



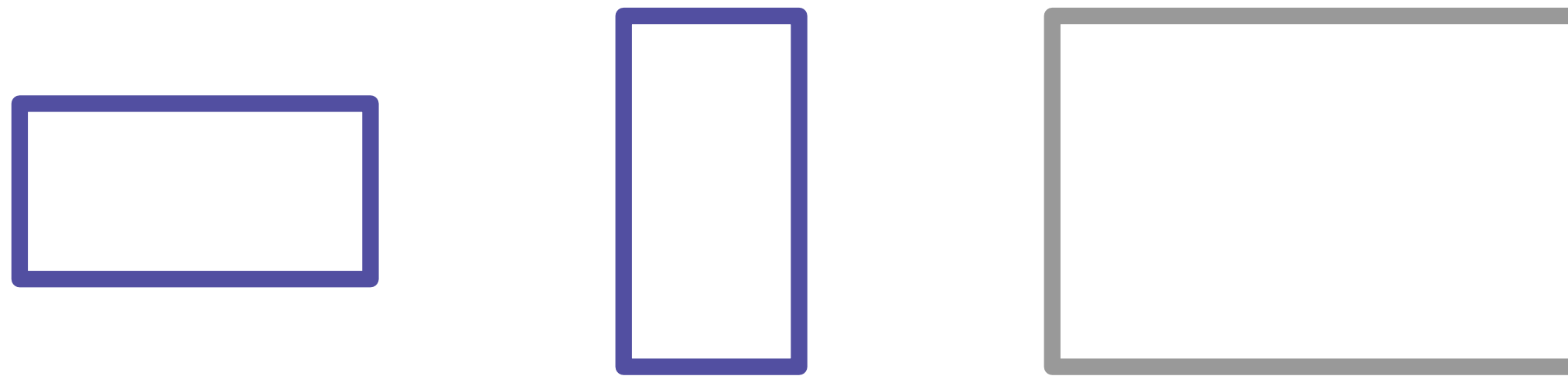
동적계획법의 문제 해결



다음과 같이 1×2 , 2×1 크기의 직사각형 모양
타일이 있을 때,



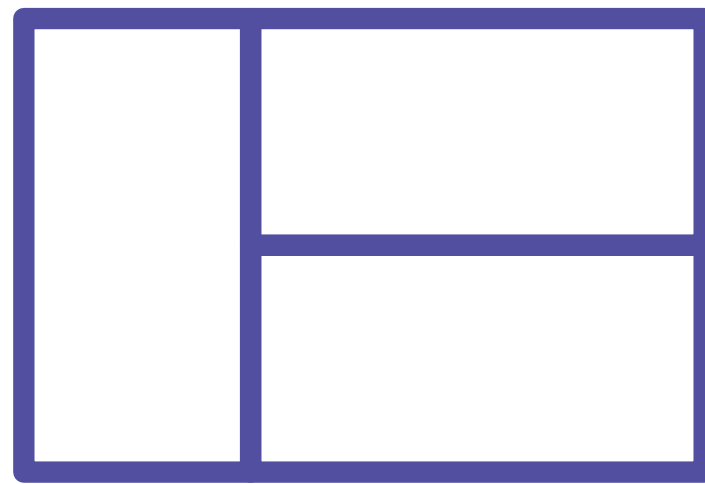
동적계획법의 문제 해결



타일을 조합하여, $2 \times N$ 크기의 벽을 채우려고 함



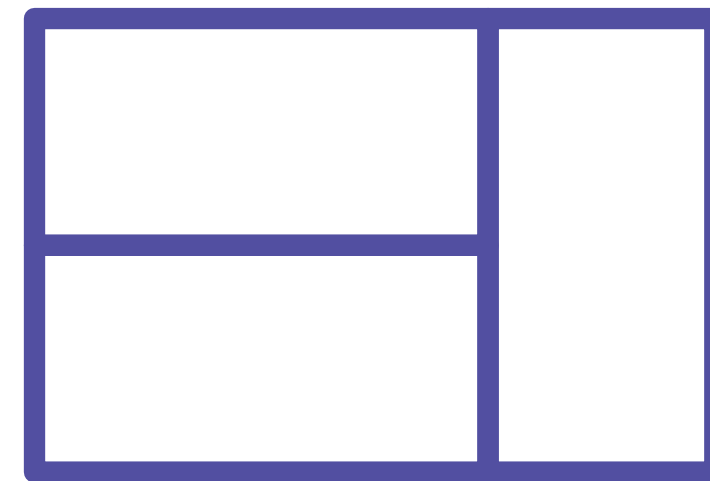
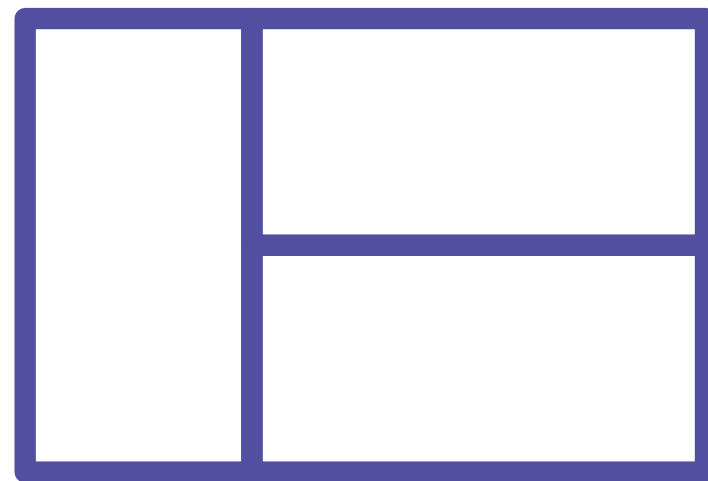
동적계획법의 문제 해결



예를 들어, 다음과 같은 방법으로
타일을 채울 수 있음



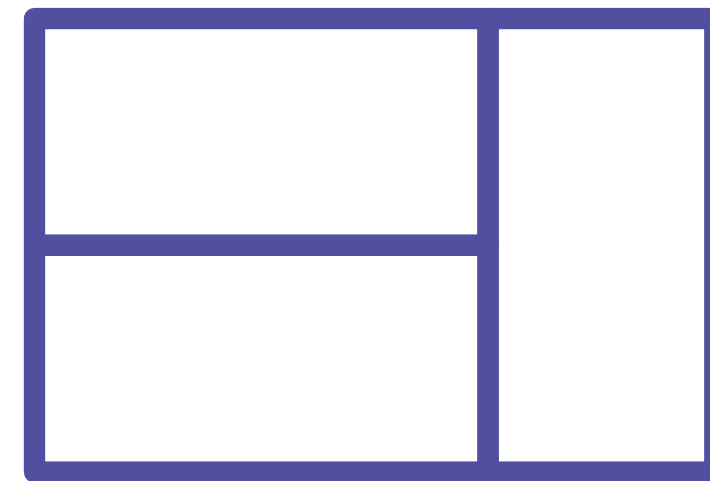
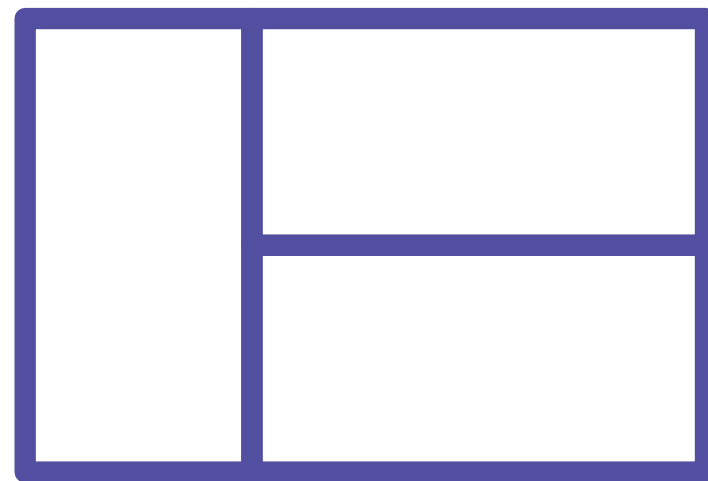
동적계획법의 문제 해결



그러나, 2×3 크기의 타일을 채울 수 있는 방법은
한 가지가 아니라, 여러 가지 방법이 있음



동적계획법의 문제 해결

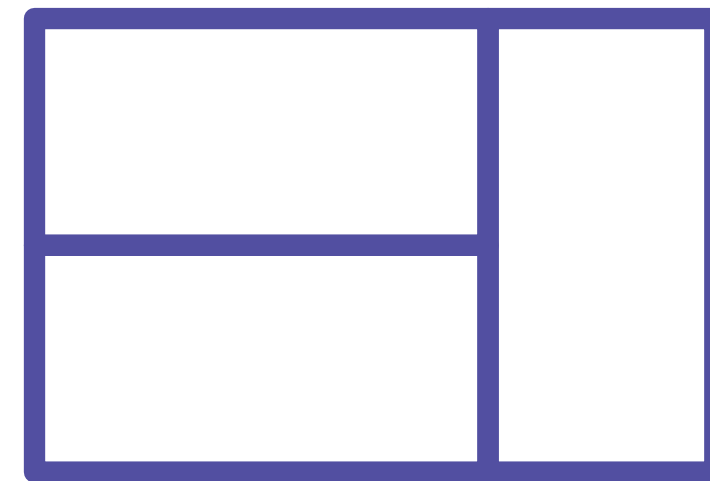
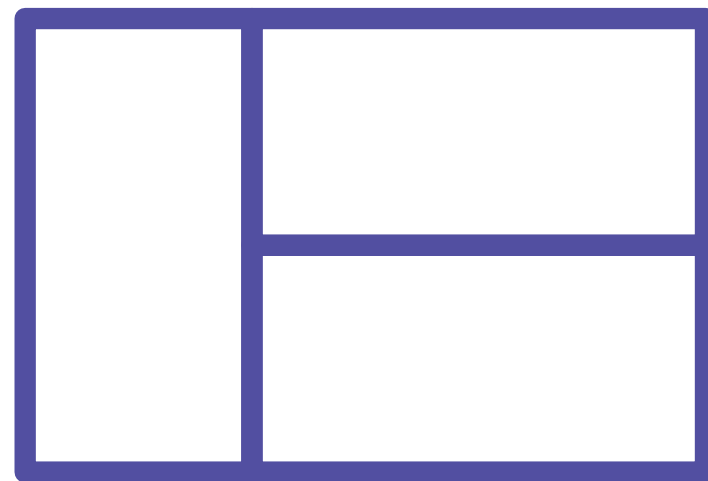


그렇다면, N 이 주어졌을 때

$2 \times N$ 크기의 벽을 채울 수 있는 경우의 수는 몇 개나 될까?



동적계획법의 문제 해결



지금까지 우리가 배웠던 알고리즘으로는
쉽게 접근할 수 없는 문제



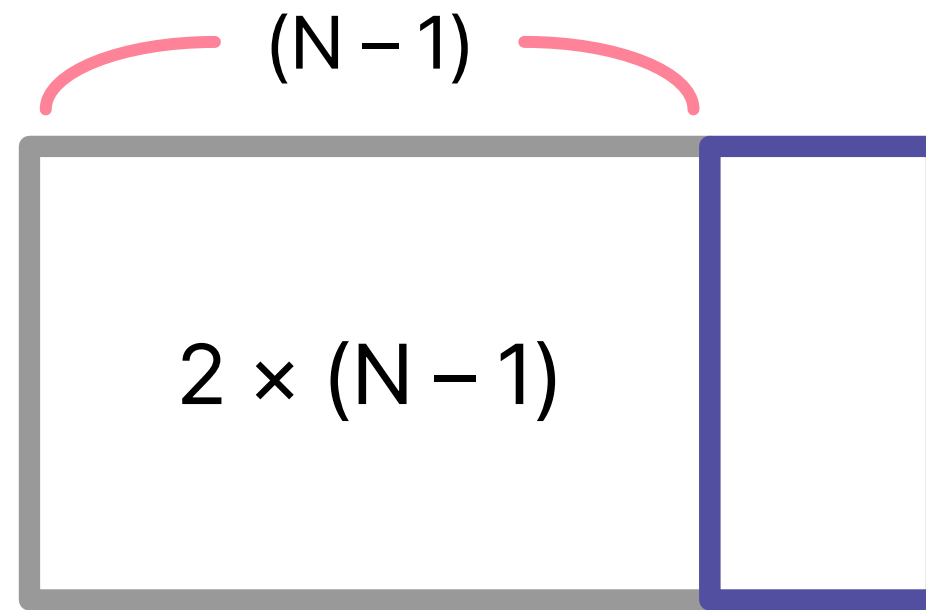
동적계획법의 문제 해결

$2 \times N$

문제를 풀기 전, 잠깐 타일의 구조를 잘 관찰해보도록 하자



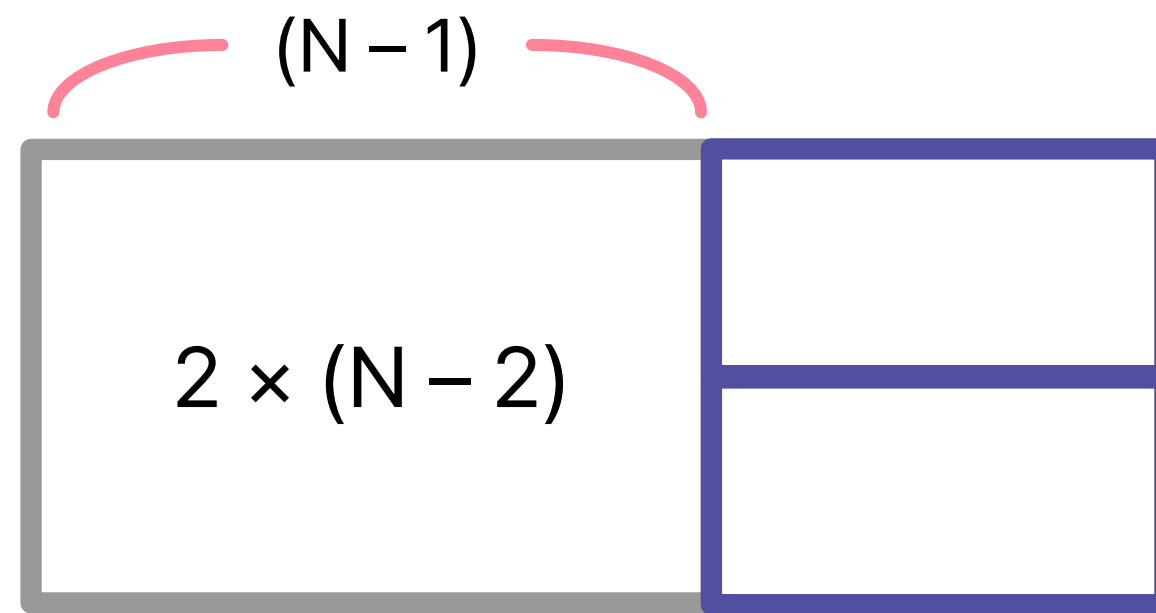
동적계획법의 문제 해결



만약 맨 끝에 세로 타일이 붙는다면,
남은 구간의 길이는 $2 \times (N - 1)$ 이 됨



동적계획법의 문제 해결

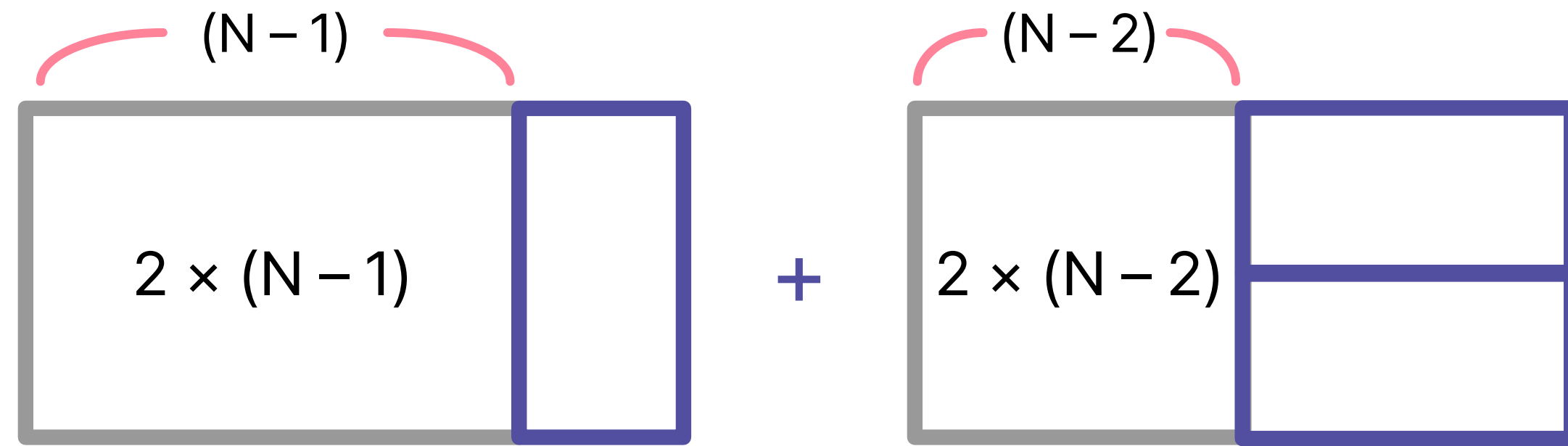


반대로, 세로 타일이 아닌 가로 타일이 끝에 붙는다면,

남은 구간의 길이는 $2 \times (N - 2)$ 가 됨



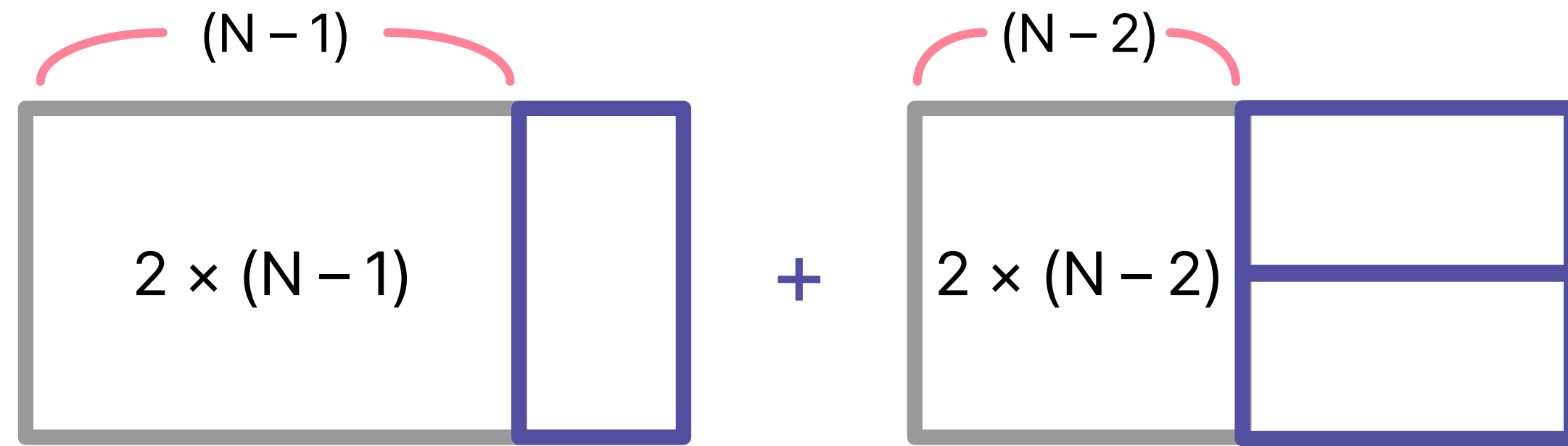
동적계획법의 문제 해결



즉, $2 \times N$ 크기의 타일을 채울 수 있는 경우의 수는
[$2 \times (N-1)$ 의 경우의 수] + [$2 \times (N-2)$ 의 경우의 수]



동적계획법의 문제 해결



간단하게, $F(N) = (2 \times N$ 크기의 타일을 채울 수 있는 경우의 수)

라고 정의하면, $F(N) = F(N-1) + F(N-2)$ 가 됨



동적계획법의 문제 해결

$$F(N) = F(N - 1) + F(N - 2)$$

식을 세웠으니, 이 식을 재귀함수로 계산하려고 하면,
중복된 부분이 많다는 것을 쉽게 파악할 수 있음



동적계획법의 문제 해결

$$F(N) = F(N - 1) + F(N - 2)$$

따라서 앞에서 배웠던 Top-down / Bottom-up 방법을 활용하여
문제를 해결할 수 있음



결국, 다이나믹 프로그래밍으로 문제를 해결하기 위해선
문제의 점화식을 잘 세워야 함



1. 구하고자 하는 값 정의하기
2. 부분문제로 표현하여 점화식 구하기
3. 코드로 옮기기



1. 구하고자 하는 값 정의하기
구하고자 하는 값이 무엇인지 정의한다
2. 부분문제로 표현하여 점화식 구하기
구하고자 하는 값을 부분문제로 구성된 식으로 표현한다
3. 코드로 옮기기
점화식을 재귀호출, 반복문으로 코드로 작성한다



그렇다면, 간단한 다이나믹 프로그래밍 문제를 풀면서
점화식을 세우는 과정을 연습해 보도록 하자!



[예제] 누적합

예시

수 N 개가 주어질 때, i 번째 수부터 j 번째 수까지의 합을 구하여라.
($1 \leq i \leq j \leq N$, N 은 10만 이하의 양의 정수)

그대로 i 부터 j 까지 단순히 그냥 더하면, 시간 복잡도 : $O(N)$

또, 이러한 합을 N 번 구해야 한다면?
(= 쿼리가 N 번 주어진다면?)
→ 시간 복잡도 : $O(N^2)$

시간 복잡도를 개선할 수는 없을까?





[예제] 누적합

- Cumulative Sum으로도 불림
- 미리 구해 놓은 배열을 통해 $O(1)$ 의 시간 복잡도로 쿼리(Query)를 해결

$$s[0] = a[0]$$

$$s[1] = a[0] + a[1]$$

$$s[2] = a[0] + a[1] + a[2]$$

...

$$s[N] = a[0] + a[1] + a[2] + \dots + a[N]$$

$$s[j] - s[i - 1] = a[i] + a[i + 1] + \dots + a[j - 1] + a[j]$$

($s[i] \rightarrow a[0]$ 부터 $a[i]$ 까지의 합)



[예제] 누적합

예시 - 1차원

- 누적합을 담을 $pSum$, 요소를 담을 arr 배열을 생성
- 계산의 편의를 위해, i 의 값을 1부터 사용
- arr 배열에 입력을 받음과 동시에 $pSum$ 배열도 갱신
 - $pSum[i - 1] \rightarrow arr[1]$ 부터 $arr[i - 1]$ 까지의 합
 - 여기에 $arr[i]$ 를 더하면, $arr[1]$ 부터 $arr[i]$ 까지의 합이 됨
- 이후, a 번째부터 b 번째까지의 합을 구하라고 하면, 다음과 같이 출력

```
import sys
input = sys.stdin.readline

# 예: 첫 줄에 N, M(질의 개수)
N, M = map(int, input().split())

arr = [0] * (N + 1)
pSum = [0] * (N + 1)

# 1-indexed 누적합
for i in range(1, N + 1):
    arr[i] = int(input())
    pSum[i] = pSum[i - 1] + arr[i]

# 구간합 질의: [a, b]
for _ in range(M):
    a, b = map(int, input().split())
    print(pSum[b] - pSum[a - 1])
```



[예제] 누적합

예시 - 2차원

- 누적합을 담을 $pSum$, 요소를 담을 arr 배열을 생성
 - $arr[i][j] \rightarrow i$ 행 j 열의 요소
 - $pSum[i][j] \rightarrow 1$ 행 1열부터 i 행 j 열까지의 합(직사각형의 형태)

```
# 1-indexed 기준 (arr, pSum 0/ (n+1) x (n+1)로 준비되어 있다고 가정)
for i in range(1, n + 1):
    for j in range(1, n + 1):
        pSum[i][j] = (
            pSum[i - 1][j] + pSum[i][j - 1] - pSum[i - 1][j - 1] + arr[i][j])
```

- arr 배열에 **입력을 받음과 동시에, $pSum$ 배열도 갱신**
 - 갱신 순서에 유의



[예제] 누적합

예시 - 2차원

```
# 1-indexed 기준 (arr, pSum) (n+1) x (n+1)로 준비되어 있다고 가정)
```

```
for i in range(1, n + 1):
```

```
    for j in range(1, n + 1):
```

```
        pSum[i][j] = (
```

```
            pSum[i - 1][j] + pSum[i][j - 1] - pSum[i - 1][j - 1] + arr[i][j])
```

$pSum[4][4]$

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

+

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

-

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

+

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1



[예제] 누적합

예시 - 2차원

- 이후, a 행 b 열부터 c 행 d 열까지의 합을 구해야 한다면?

```
# (a, b) ~ (c, d) 직사각형 구간 합 (1-indexed)
ans = pSum[c][d] - pSum[c][b - 1] - pSum[a - 1][d] + pSum[a - 1][b - 1]
print(ans)
```

(a, b)

1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5

(c, d)



[예제] 누적합

예시 - 2차원

(1, 1)

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

(5, 5)

파란색 부분을 구하려면?

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

-

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

-

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1

+

1	2	3	4	5
1	5	1	4	3
2	5	3	3	1
3	2	5	4	3
2	4	4	5	1



누적합의 장단점

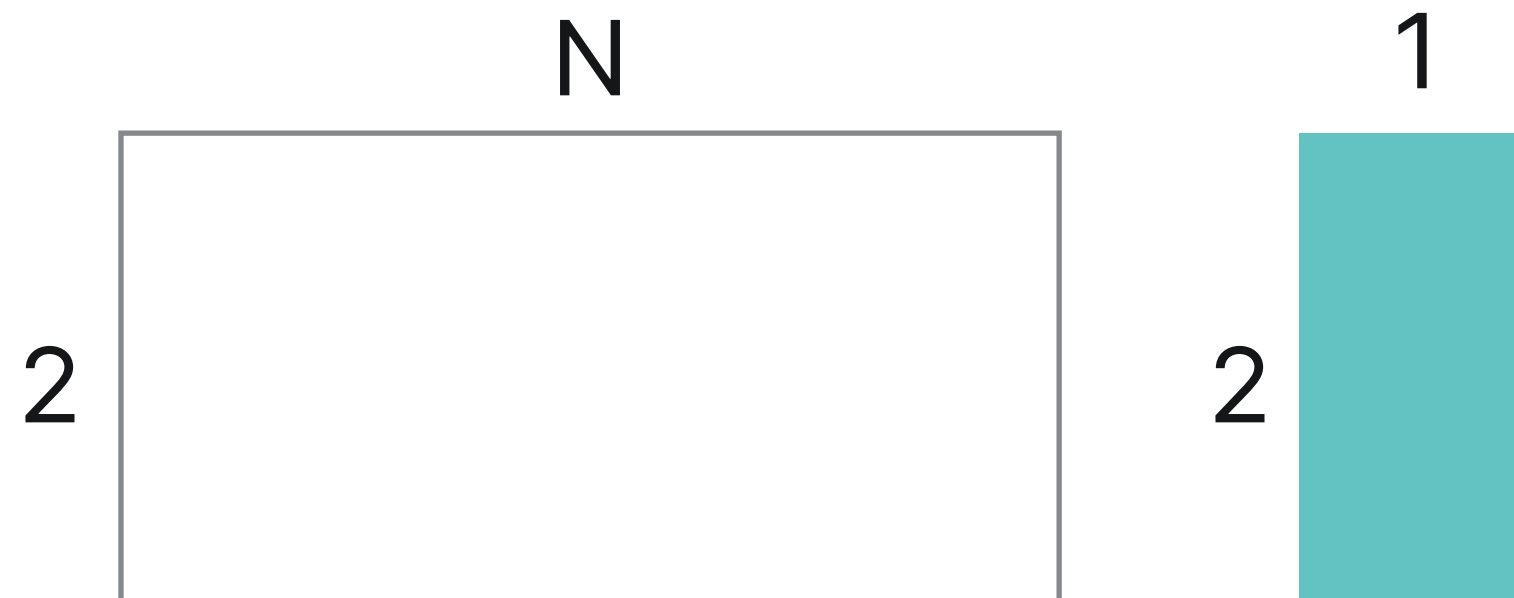
- 빠른 구간 합 계산이 가능
- 상대적으로 간단한 구현
- 데이터 변경의 어려움
 - 데이터가 변경되면, 변경될 때마다 구간합을 다시 계산해야 함
- 메모리 사용
 - 구간합을 계산하여 저장해야 하므로, 추가적인 메모리가 필요

누적합 문제는 주로 구간 합 구하기 문항에서 자주 출제



[예제] 타일 채우기

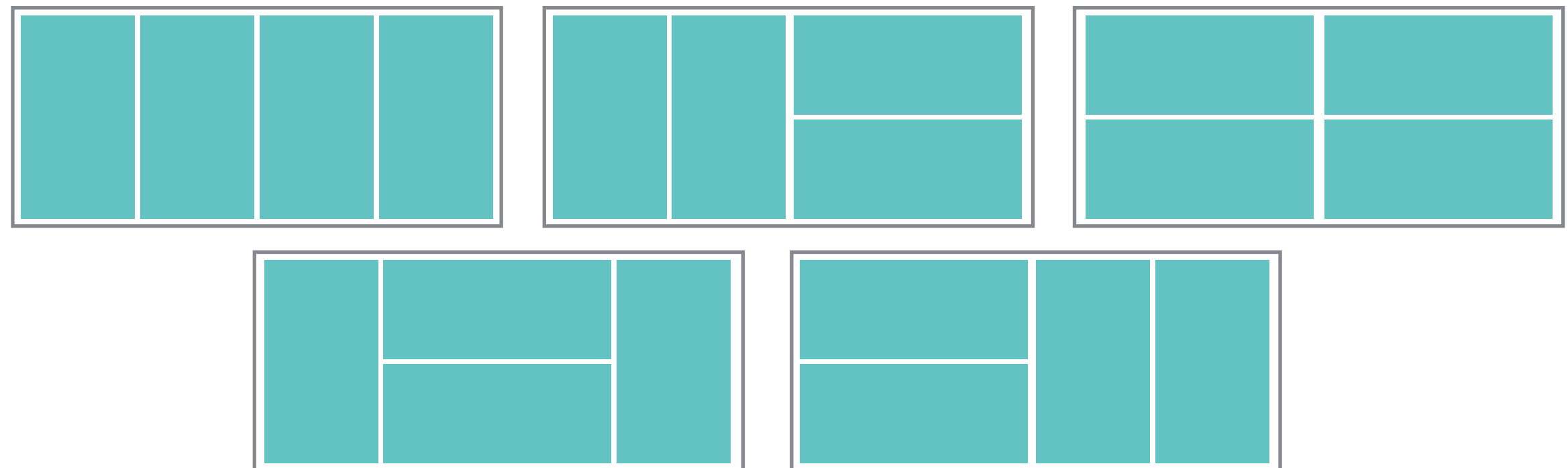
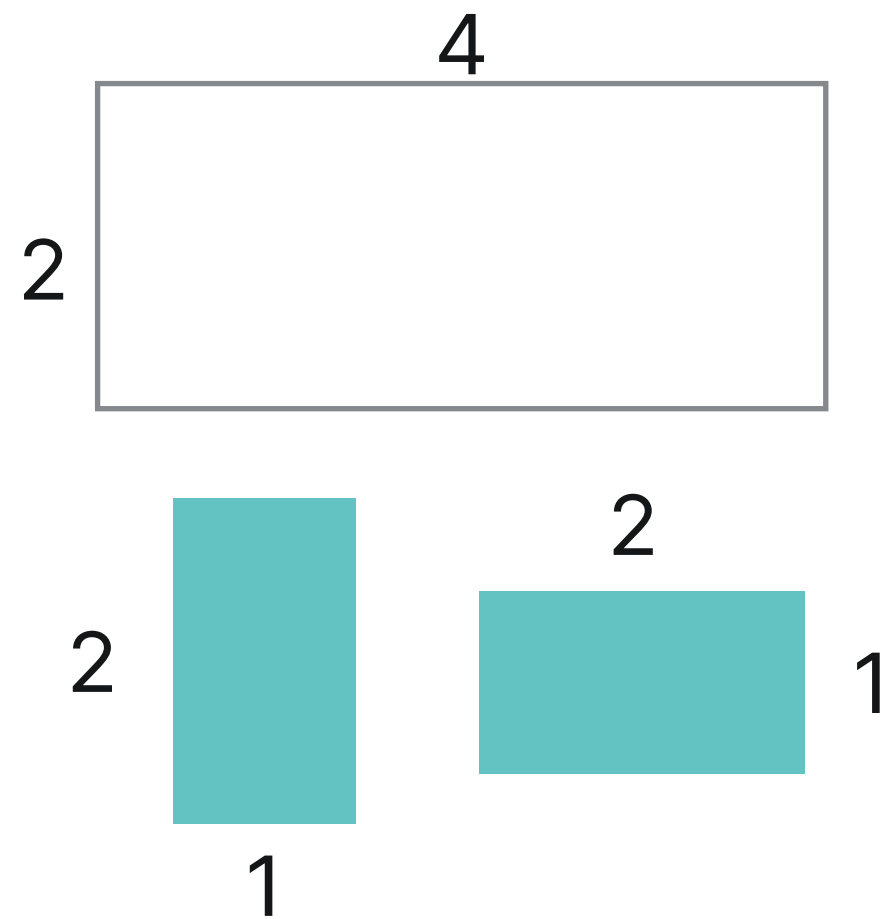
2xN짜리 액자에 2x1크기의 타일을 채워 넣으려고 합니다.
해당 액자와 타일로 만들 수 있는 **서로 다른 작품의 수는 몇 개**인가요?





[예제] 타일 채우기

2xN짜리 액자에 2x1크기의 타일을 채워 넣으려고 합니다.
해당 액자와 타일로 만들 수 있는 **서로 다른 작품의 수는 몇 개**인가요?





[예제] 타일 채우기

1. 구하고자 하는 값 정의하기
구하고자 하는 값이 무엇인지 정의한다

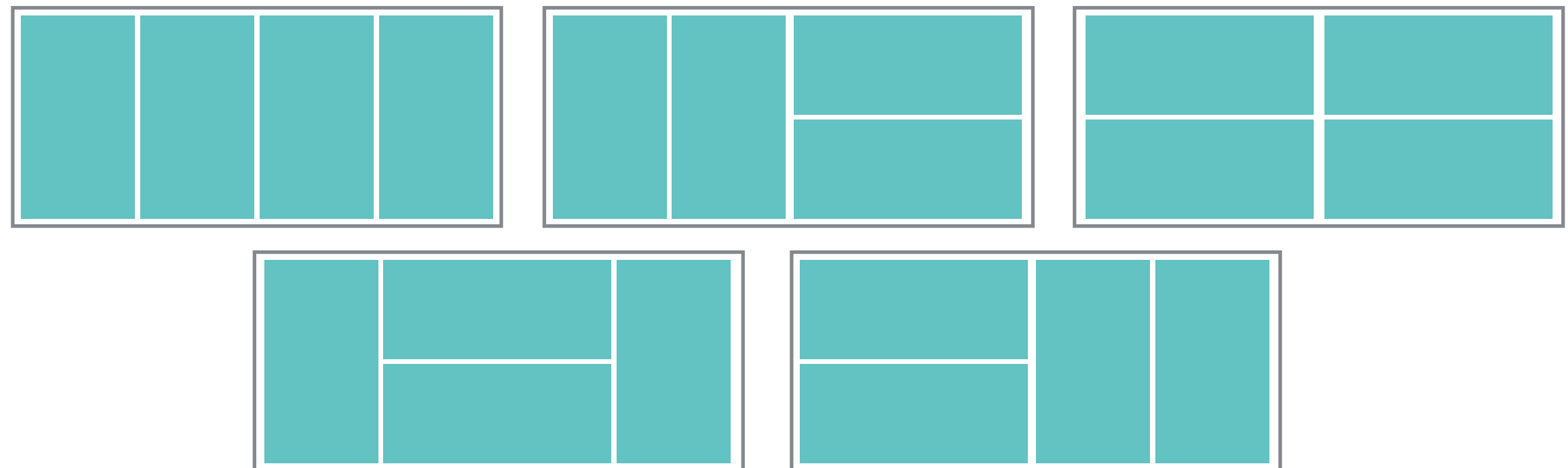
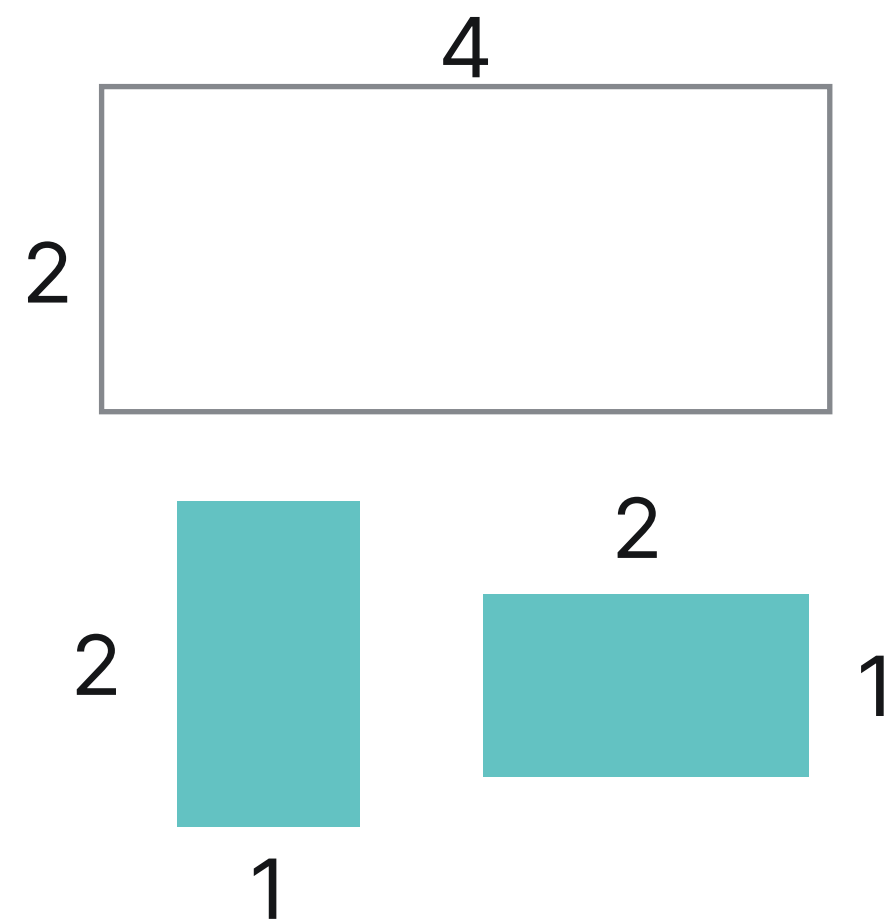
$T(n) = 2 \times n$ 크기의 액자를 채우는 경우의 수



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

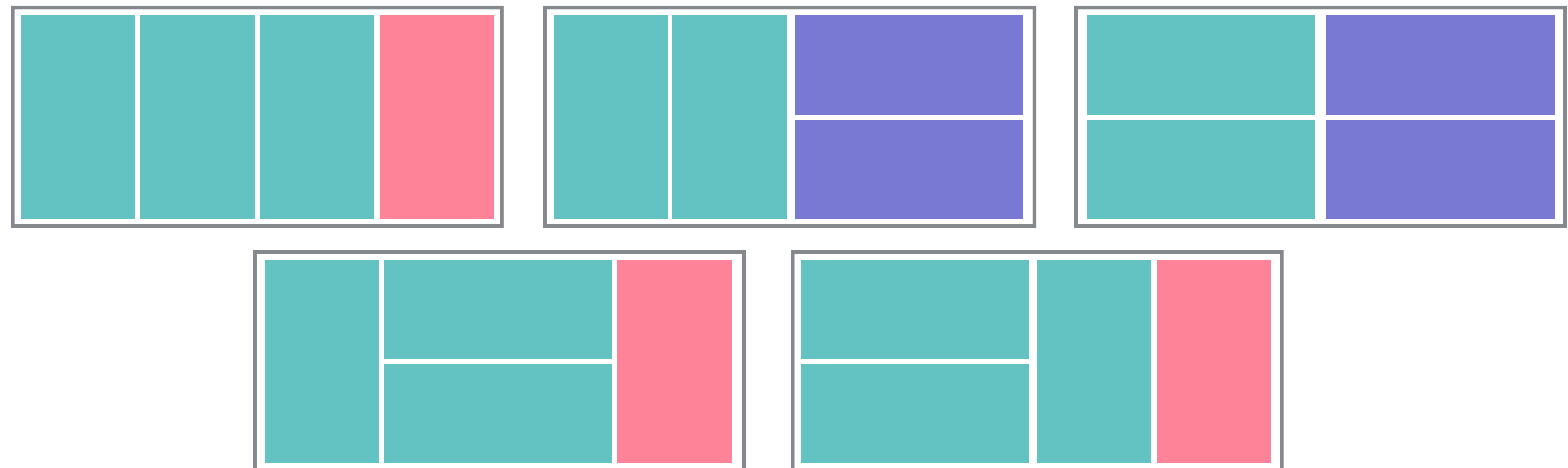
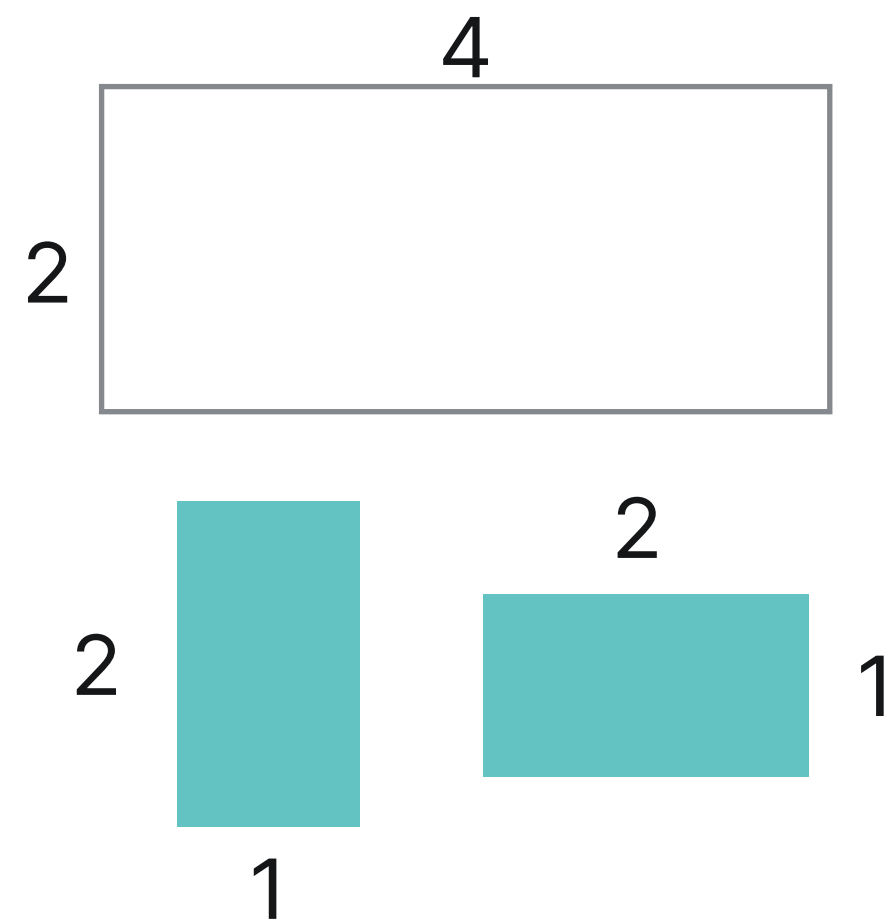




[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다





[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

1



$2 \times (n-1)$ 의 타일을 채우는 경우의 수
 $T(n-1)$

2



$2 \times (n-2)$ 의 타일을 채우는 경우의 수
 $T(n-2)$



2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1							



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1	1						



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1	1	2					



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1	1	2	3				



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1	1	2	3	5			



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1	1	2	3	5	8		



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1	1	2	3	5	8	13	



[예제] 타일 채우기

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$T(n) = 2 \times n$ 의 액자를 채우는 경우의 수

$$T(n) = T(n-1) + T(n-2)$$

	0	1	2	3	4	5	6	7
T	1	1	2	3	5	8	13	21



3. 코드로 옮기기

점화식을 재귀호출, 반복문 식으로 코드로 작성한다

$$T(n) = T(n-1) + T(n-2)$$

```
def T(n):  
    if n == 0 or n == 1: return 1  
  
    if n in memo: return memo[n]  
    else:  
        memo[n] = T(n-1) + T(n-2)  
  
    return memo[n]
```



[예제] 타일 채우기

3. 코드로 옮기기

점화식을 재귀호출, 반복문 식으로 코드로 작성한다

$$T(n) = T(n-1) + T(n-2)$$

```
T = [1, 1]
```

```
for i in range(2, n+1):  
    T.append(T[i-1] + T[i-2])
```



[예제] 숫자 만들기

1부터 M까지의 숫자를 사용하여 합이 N이 되도록 만드는 경우의 수는?

(1+2와 2+1은 서로 다른 경우입니다.)

예) 1부터 3까지 숫자를 이용해서 (M=3) 합이 5 (n=5)가 되도록 만드는 경우의 수는 모두 13가지입니다.

$$1 + 1 + 1 + 1 + 1$$

$$1 + 1 + 1 + 2$$

$$1 + 1 + 2 + 1$$

$$1 + 2 + 1 + 1$$

$$2 + 1 + 1 + 1$$

$$1 + 2 + 2$$

$$2 + 1 + 2$$

$$2 + 2 + 1$$

$$1 + 1 + 3$$

$$1 + 3 + 1$$

$$3 + 1 + 1$$

$$2 + 3$$

$$3 + 2$$



[예제] 숫자 만들기

1. 구하고자 하는 값 정의하기
구하고자 하는 값이 무엇인지 정의한다

$\text{sum_N}(n)$ = 1 ~ M 의 수를 이용하여 n 를 만드는 경우의 수



2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

1 ~ M 의 수를 이용하여 n 을
만드는 경우의 수

=

(n-1)을 만드는 경우의 수 +
(n-2)을 만드는 경우의 수 +
(n-3)을 만드는 경우의 수 +
... +
(n-M)을 만드는 경우의 수



2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$\text{sum_N}(n)$ = 1 ~ M 의 수를 이용하여 n 을 만드는 경우의 수

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N								



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1							



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1	1						



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1	1	2					



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1	1	2	4				



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1	1	2	4	7			



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1	1	2	4	7	13		



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1	1	2	4	7	13	24	



[예제] 숫자 만들기

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

N=7

M=3

	0	1	2	3	4	5	6	7
Sum_N	1	1	2	4	7	13	24	44



[예제] 숫자 만들기

3. 코드로 옮기기

점화식을 재귀호출, 반복문 식으로 코드로 작성한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

```
def sum_N(n):  
    if n == 0: return 1  
    if n in memo:  
        return memo[n]
```

```
    else:  
        sum_temp = 0  
  
        for i in range(1, M+1)  
            sum_temp += sum_N(n-i)  
        memo[n] = sum_temp  
    return memo[n]
```



3. 코드로 옮기기

점화식을 재귀호출, 반복문 식으로 코드로 작성한다

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

```
sum_N = [1]
for n in range(1, N+1):
    sum_temp = 0
    for i in range(1, M+1)
        sum_temp += sum_N[n-i]
    sum_N.append(sum_temp)
```



3. 코드로 옮기기

점화식을 재귀호출, 반복문 식으로 코드로 작성한다

$n < M$ 이라면?

$$\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$$

```
sum_N = [1]
for n in range(1, N+1):
    sum_temp = 0
    for i in range(1, M+1)
        sum_temp += sum_N[n-i]
    sum_N.append(sum_temp)
```




3. 코드로 옮기기

점화식을 재귀호출, 반복문 식으로 코드로 작성한다

$n < M$ 일 경우, $\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-n)$

$n \geq M$ 일 경우, $\text{sum_N}(n) = \text{sum_N}(n-1) + \text{sum_N}(n-2) + \dots + \text{sum_N}(n-M)$

```
sum_N = [1]
for n in range(1, N+1):
    sum_temp = 0
    for i in range(1, min(n, M)+1):
        sum_temp += sum_N[n-i]
    sum_N.append(sum_temp)
```



[예제] 짜장, 짬뽕, 볶음밥!

상훈이는 점심엔 늘 짜장, 짬뽕, 볶음밥 셋 중 하나를 먹는다. 하지만 거기엔 규칙이 하나 있는데, **전날 먹은 음식은 먹지 않는다**는 것이다. 예를 들어 어제 짜장면을 먹었으면, 오늘은 짬뽕이나 볶음밥 중 하나를 먹어야 하는 것이다.

짜장, 짬뽕, 볶음밥 선호도는 **그날그날** 다른데, 매일 선호도가 주어질 때, 적절히 날마다 음식을 결정하여 **총 선호도의 최대값**을 구하시오.

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

총 선호도 116



[예제] 짜장, 짬뽕, 볶음밥!

1. 구하고자 하는 값 정의하기
구하고자 하는 값이 무엇인지 정의한다

$\text{Delicious}(i)$ = i 번째 날까지 구한 최대 선호도 합



[예제] 짜장, 짬뽕, 볶음밥!

1. 구하고자 하는 값 정의하기
구하고자 하는 값이 무엇인지 정의한다

짜장: 0, 짬뽕: 1, 볶음밥: 2

$\text{Delicious}(i, j)$ = i 번째 날에 j 음식을 먹었을 때 지금까지의 최대 선호도합



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

i 번째 날



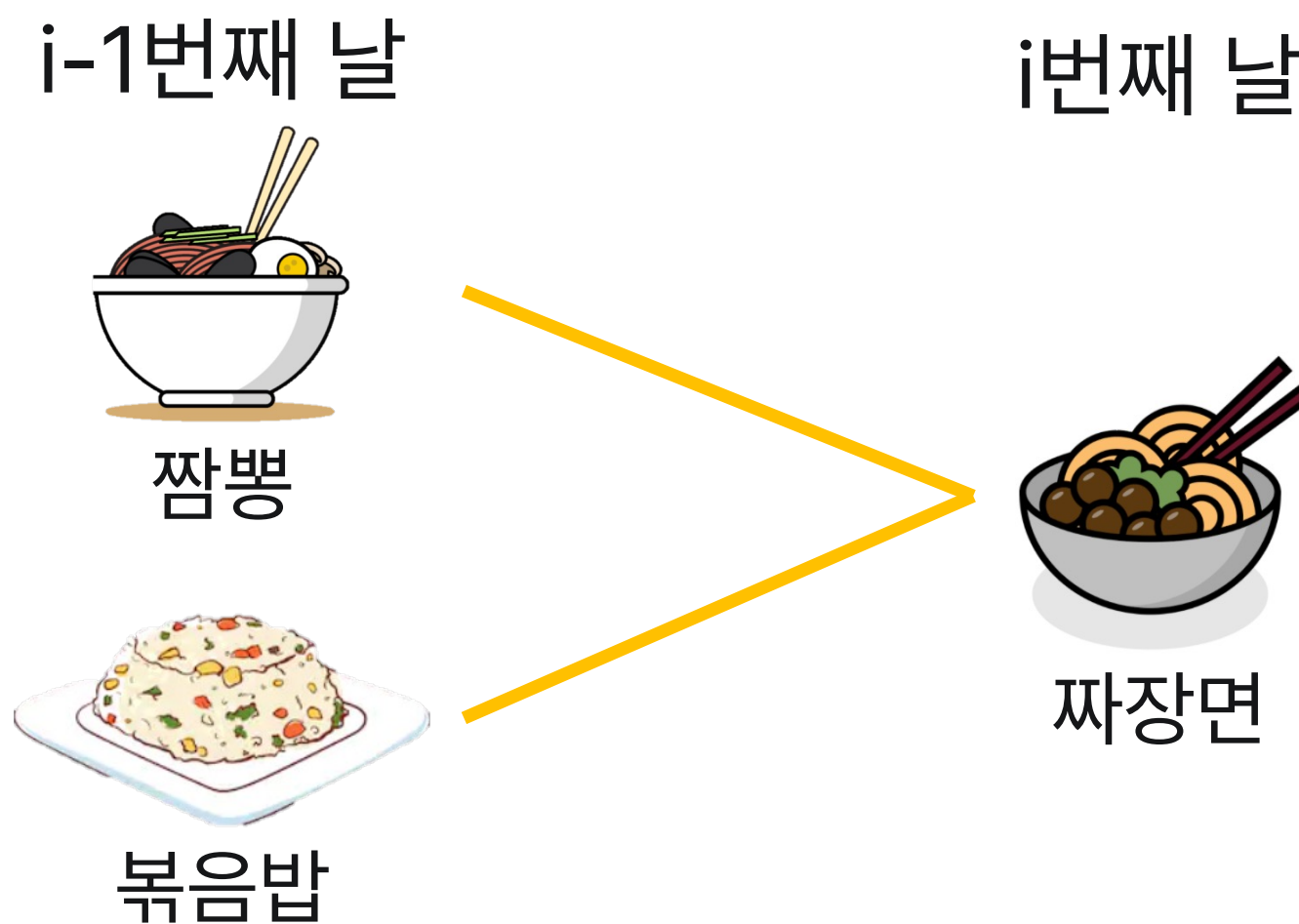
짜장면



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

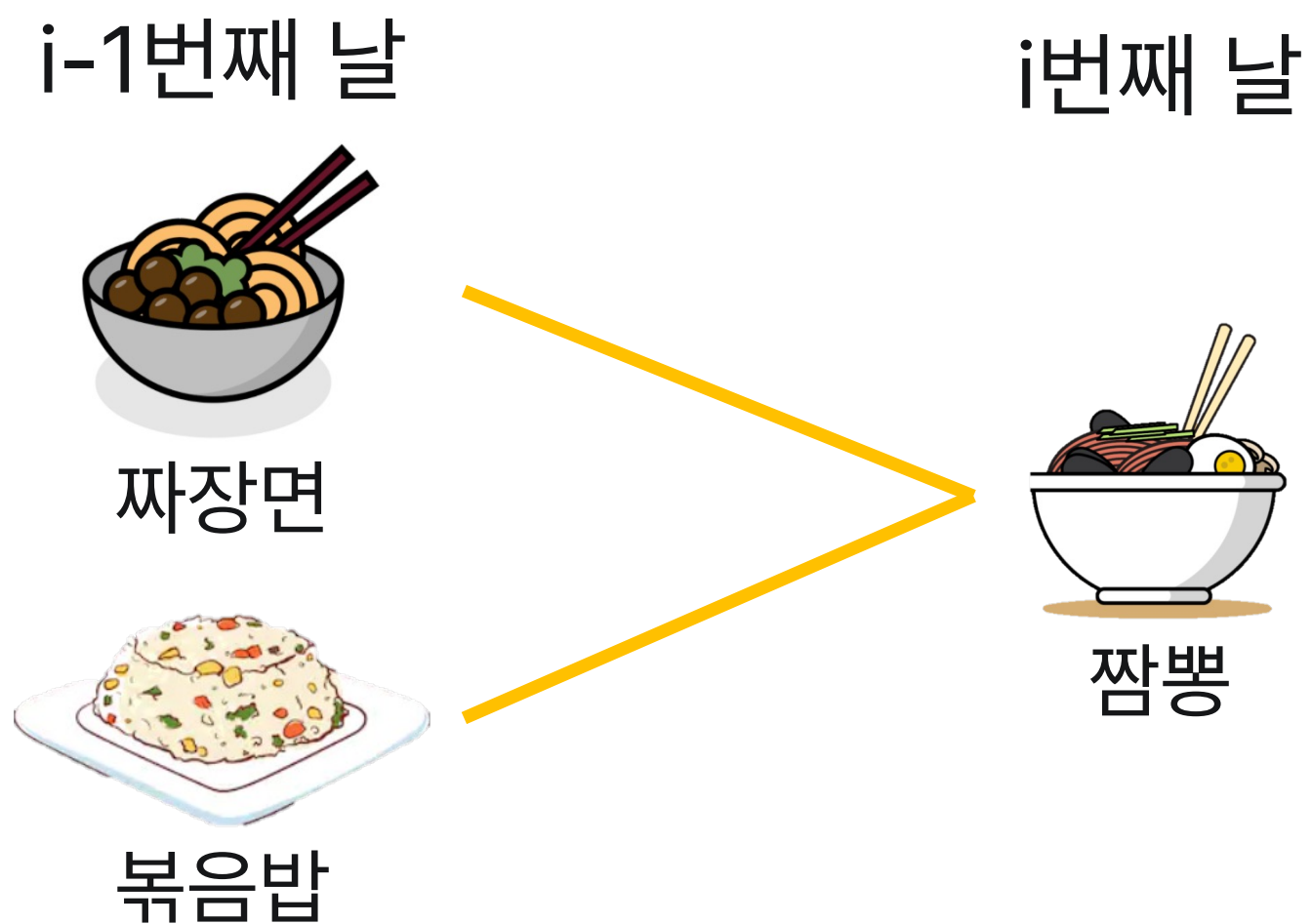




[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다





[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

i-1번째 날



짜장면



짬뽕

i번째 날



볶음밥



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일			
2일			
3일			



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27		
2일			
3일			



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	
2일			
3일			



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일			
3일			



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	53		
3일			



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	53	71	
3일			



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	53	71	37
3일			



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	53	71	37
3일	78		



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	53	71	37
3일	78	75	



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	53	71	37
3일	78	75	116



[예제] 짜장, 짬뽕, 볶음밥!

2. 부분문제로 표현하여 점화식 구하기 - 동작 확인

구하고자 하는 값을 부분문제로 구성된 식으로 표현한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

preference

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	18	36	10
3일	7	22	45

Delicious

구분	짜장	짬뽕	볶음밥
1일	27	8	35
2일	53	71	37
3일	78	75	116



[예제] 짜장, 짬뽕, 볶음밥!

3. 코드로 옮기기

점화식을 재귀호출, 반복문 식으로 코드로 작성한다

$$\text{Delicious}(i, j) = \max(\text{Delicious}(i-1, k)) + \text{preference}(i, j) \{k \neq j\}$$

```
for i in range(1, days+1):  
    for j in range(3):  
        for k in range(3):  
            if j == k: continue  
            Delicious[i][j] = max(Delicious[i][j], Delicious[i-1][k])  
        Delicious[i][j] += preference[i][j]
```